Exception Handling in Java

Exception Handling is the mechanism used to handle **runtime exceptions using try, catch, finally, throw, and throws,** to maintain the normal flow of **program execution.**

Java exceptions are mainly classified into two types:

1. Checked Exceptions
2. Unchecked Exceptions

➤ Checked Exceptions:

Checked exceptions are exceptions that are **checked by the compiler at compile time**.

The developer must **handle them using try-catch** or **declare them using throws** to avoid compilation errors.

Examples:

- IOException
- SQLException
- FileNotFoundException
- ClassNotFoundException

➤ Un – Checked Exceptions:

Unchecked exceptions are exceptions that are **not checked by the compiler at compile time** and generally occur during runtime.

Therefore, handling them is **optional at compile time** because the Java compiler **does not verify whether they are handled or declared**.

Examples:

- **ArithmeticException**
- **NullPointerException**
- **ArrayIndexOutOfBoundsException**
- **NumberFormatException**

➥ Exception:

❀ ——————————————————————————————————— ❀

An **exception is an event that occurs during program execution** and disrupts the normal flow of the program.

Example: ArithmeticException.

➥ Try block:

❀ ——————————————————————————————————— ❀

The try block contains **risky code** that may **generate an exception**.

Syntax:
```
try {
        //Statements.
}
```

➥ Catch block:

❀ ——————————————————————————————————— ❀

The catch block is used to **handle the exception** thrown by the try block.

Syntax:
```
try {
        //risky code
} catch (ExceptionType variable){
        //handling code
}
```

Examples:

```
1  package com.channa.JavaPractice;
2
3  public class Demo1 {
       Run | Debug
4      public static void main(String[] args) {
5          try{
6              int result = 10/0;
7              System.out.println("Result : " + result);
8          } catch(ArithmeticException e){
9              System.out.println(x: "Cannot divide by Zero.");
10         }
11         System.out.println(x: "Program continues...");
12     }
13 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclipse Adop
tium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMe
ssages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5
e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com
.channa.JavaPractice.Demo1'
Cannot divide by Zero.
Program continues...
PS C:\Users\chann\OneDrive\Desktop\Java>
```

IMP: **ArithmeticException** = exception class type.
e = reference variable holding exception object.

Rule: A **try block** must be followed by **at least one catch or finally.**

➥ Multiple Catch blocks:

❀ ——————————————————————————————————— ❀

Multiple catch blocks are used to **handle different types of exceptions separately** using a **single try block**.

Syntax:

```
trye {
        //Risky code.
} catch(ExceptionType1 variable){
        //handling code
} catch(ExceptionType2 variable){
        //handling code
}
```

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    Run | Debug
    public static void main(String[] args) {
        try{
            int [] arr = {10, 20, 30};//only one exception occurs at a time in a single execution of try block.
            System.out.println(arr[5]);
            int res = 10 / 0;
            System.out.println(res);
        } catch (ArithmeticException e){
            System.out.println(x: "Cannot divide by Zero.");
        } catch (ArrayIndexOutOfBoundsException e){
            System.out.println(x: "Invalid Array Index");
        }
        System.out.println(x: "Program continues...");
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Progr
am Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\ja
va.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-c
p' 'C:\Users\chann\AppData\Roaming\Code\User\workspac
eStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java
\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.D
emo1'
Invalid Array Index
Program continues...
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Note: In the above example arr[5] generates ArrayIndexOutOfBoundException. So, JVM immediately jumps to that matching catch block. The division code will never execute.

➥ Exception Hierarchy:

❀ ——————————————————————————————————— ❀

Object
↓
Throwable //Root class

- **Exception** // parent class for RuntimeException
  - **RuntimeException** //parent class for all specific RuntimeExceptions.
    - **ArithmeticException**
    - **NullPointerException**
    - **ArrayIndexOutOfBoundsException**
    - **NumberFormatException**
- **Other Checked Exceptions**
  - **IOException**
  - **SQLException**
  - **ClassNotFoundException**
- **Error**
  - **OutOfMemoryError**
  - **StackOverflowError**

Throwable is the root class for **exceptions and errors**.

- **Exception class** represents conditions that applications can generally **handle**.
- **RuntimeException** class and its subclasses are **unchecked exceptions**.
- Other subclasses of **Exception** are generally **checked exceptions**.
- And **Error** represents serious problems that applications generally **should not try to handle**.

➤ Finally Block:

The finally block is used to execute **cleanup or essential code** after the try and catch blocks, regardless of whether an exception occurs or is handled.

finally is mainly used for **cleanup operations**, such as **closing files, database connections, or other resources**.

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    Run | Debug
    public static void main(String[] args) {
    try{
            int res = 10 / 0;
            System.out.println(res);
        } catch (ArithmeticException e){
            System.out.println(x: "Cannot divide by Zero.");
        } finally{
            System.out.println(x: "Always Runs");
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Cannot divide by Zero.
Always Runs
PS C:\Users\chann\OneDrive\Desktop\Java>
```

➤ Throw Statement:

The throw statement is used to **explicitly throw an exception** at a specific point in the program.

Syntax: throw new ExceptionType (“Message”);

Example:

Note: **throw** is not a part of the exception hierarchy.

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    Run | Debug
    public static void main(String[] args) {
        int age = 17;
        if(age < 18) {
            throw new IllegalArgumentException(s: "Not eligible for vote");
        }
        System.out.println(x: "Eligible for vote");
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Exception in thread "main" java.lang.IllegalArgumentException: Not eligible for vote
        at com.channa.JavaPractice.Demo1.main(Demo1.java:7)
PS C:\Users\chann\OneDrive\Desktop\Java>
```

➤Throws Statement:

❀——————————————————————————————❀

The throws keyword is used to **declare one or more exceptions that a method may throw** and **propagates the responsibility of handling those exceptions to the caller**.

Note: Exception propagates: The **exception is passed from the current method to the calling method** for handling.

Syntax:
```
returnType methodName() throws ExceptionType {
        //Statements.
}
```

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    static void checkAge(int age) throws Exception{
        if(age < 18){
            // Exception propagates to the caller to handle it.
            throw new Exception(message: "Not Eligible for vote");
        }System.out.println(x: "Eligible for vote");
    }
    Run | Debug
    public static void main(String[] args) {
        try{
            checkAge(age: 15);
        } catch (Exception e){ //exception handled.
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclipse Adoptium
\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
'-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f
5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPrac
tice.Demo1'
Not Eligible for vote
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Understanding flow:

throw

↓

exception occurs

↓

not handled in checkAge()

↓

propagates to main()

↓

catch handles it

↓

getMessage()

↓

prints "Not Eligible for vote"

Note: Unchecked exception can propagate without throws.

⤷➤ Exception Propagation:

❀ ——————————————————————————————— ❀

Exception propagation is the process of **passing an unhandled exception from the current method to its caller** until a suitable catch block handles it.

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    static void checkAge(int age) throws Exception{
        if(age < 18){
            // Exception propagates to the caller to handle it.
            throw new Exception(message: "Not Eligible for vote");
        }System.out.println(x: "Eligible for vote");
    }
    Run | Debug
    public static void main(String[] args) {
        try{
            checkAge(age: 15);
        } catch (Exception e){ //exception handled.
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclipse Adoptium
\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
'-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f
5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPrac
tice.Demo1'
Not Eligible for vote
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Simple flow:

Current Method
↓
Exception occurs
↓
Not handled here
↓
Propagates to Caller
↓
Caller has catch?
↓
YES → Exception handled

Notes: **throws** declares that an exception may propagate to the caller.

Exception propagation is classified into two types:

1. Checked Exception Propagation.
2. Unchecked Exception Propagation.

⤷➤ Checked Exception Propagation:

❀ ——————————————————————————————— ❀

Checked exception propagation is the process of **passing an unhandled checked exception from one method to another** using the **throws keyword** until it is handled using a **try-catch** block.

Example:

```java
package com.channa.JavaPractice;

public class Demo1 {
    static void checkAge(int age) throws Exception{
        if(age < 18){
            // Exception propagates to the caller to handle it.
            throw new Exception(message: "Not Eligible for vote");
        }System.out.println(x: "Eligible for vote");
    }
    Run | Debug
    public static void main(String[] args) {
        try{
            checkAge(age: 15);
        } catch (Exception e){ //exception handled.
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclip
se Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\Us
er\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jd
t_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Not Eligible for vote
PS C:\Users\chann\OneDrive\Desktop\Java>
```

⤷➤ Un – Checked Exception Propagation:

❀ ——————————————————————————————— ❀

Unchecked exception propagation is the process of **passing an unchecked exception from one method to another method automatically** until it is handled using a **try-catch** block.

Example:



```java
package com.channa.JavaPractice;

public class Demo1 {
    static void checkAge(int age){
        if(age < 18){
            throw new IllegalArgumentException(s: "Not Eligible for vote");
        }System.out.println(x: "Eligible for vote");
    }
    Run | Debug
    public static void main(String[] args) {
        try{
            checkAge(age: 15);
        } catch (Exception e){ //exception handled.
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclip
se Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\Us
er\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jd
t_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Not Eligible for vote
PS C:\Users\chann\OneDrive\Desktop\Java>
```

⤷➤ Custom Exception:

❀ ——————————————————————————————— ❀

A custom exception is a user-defined exception created by extending an **existing exception class.**

Custom exceptions are widely used in production applications because **built-in exceptions cannot always clearly describe specific business requirements or application-specific errors.**

Example:



```java
package com.channa.JavaPractice;

class InsufficientBalanceException extends Exception {

    InsufficientBalanceException(String message) {
        super(message);
    }
}
public class Demo1 {
    static void withdraw(double balance, double amount)
            throws InsufficientBalanceException {

        if (amount > balance) {
            throw new InsufficientBalanceException(message: "Insufficient balance for withdrawal");
        }
        System.out.println(x: "Withdrawal successful");
    }
    Run | Debug
    public static void main(String[] args) {
        try {
            withdraw(balance: 5000, amount: 7000);

        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclip
se Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\Us
er\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jd
t_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Insufficient balance for withdrawal
PS C:\Users\chann\OneDrive\Desktop\Java>
```

➥ getMessage():

❀ ——————————————————————————————— ❀

getMessage() is a method of the **Throwable class** used to **retrieve the message associated with an exception**.

Example:

```java
package com.channa.JavaPractice;

class InsufficientBalanceException extends Exception {

    InsufficientBalanceException(String message) {
        super(message);
    }
}
public class Demo1 {
    static void withdraw(double balance, double amount)
            throws InsufficientBalanceException {

        if (amount > balance) {
            throw new InsufficientBalanceException(message: "Insufficient balance for withdrawal");
        }
        System.out.println(x: "Withdrawal successful");
    }
    Run | Debug
    public static void main(String[] args) {
        try {
            withdraw(balance: 5000, amount: 7000);

        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java>  & 'C:\Program Files\Eclip
se Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\Us
er\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jd
t_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Insufficient balance for withdrawal
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Here, the message: **"Insufficient balance for withdrawal"** is passed to the **custom exception constructor: super(message)** passes it to the **Exception class,** which stores the message.

Then the exception propagates to main(): **catch (InsufficientBalanceException e)**

Here, **e** refers to the same exception object.

Finally: **e.getMessage()** retrieves the message stored in that **exception object.**

Difference between throw, throws, and Throwable

Throw	Throws	Throwable
throw is a keyword used inside a **method/block to explicitly throw an exception.**	**throws** is a keyword used **in a method declaration to declare and propagate an exception to the caller.**	**Throwable is a class that represents the root of Java's throwable hierarchy and is the superclass of Exception and Error.**
throw throws one exception at a time.	**throws** can declare multiple exceptions: **void** test() **throws IOException, SQLException**	**Not applicable.**

Difference between final, finally and finalize()

final	finally	finalize ()
final is a **keyword** used to **restrict modification.**	**finally** is a **block of code** used with **try/catch** for **cleanup activities** that is generally executed **whether an exception occurs or not**.	**finalize()** is a **deprecated method** that was **intended (designed)** for **cleanup before garbage collection**.

➤ Error in Java:

An **error** is a problem in a program that causes it to **fail to compile, produce incorrect output, or fail during execution**.

There are **three common types of errors** in general programming:

1. **Syntax error**
2. **Logical error**
3. **Runtime error**

➤ Syntax error:

A **Syntax Error** occurs when the program violates the **syntax or rules of the programming language**.

It is usually detected during **compilation**.

Example:

```
int a = 10
System.out.println(a);
```

Here, the semicolon ; is missing after 10, so the compiler reports an error.

Key point: Syntax errors are usually detected **before the program runs**.

⤷➤ Logical error:

❀ ——————————————————————————————————— ❀

A **Logical Error** occurs when the program **runs successfully but produces the wrong output** because the logic of the program is incorrect.

Example:

```
int a = 10;
int b = 20;
System.out.println(a - b);
```

If the intention was to calculate the sum, the program produces the wrong result because the **logic is incorrect**.

Key point: The program **runs**, but the **output is incorrect**.

⤷➤ Runtime error:

❀ ——————————————————————————————————— ❀

A **Runtime Error** is a problem that occurs **while the program is executing**.

Example:

```
int a = 10;
int b = 0;
System.out.println(a / b);
This causes: ArithmeticException
```

because division by zero is not allowed.

Key point: Runtime errors occur **during program execution**.

Note: Runtime errors caused by exceptions can generally be handled using try-catch.

🟤 **JAVA EXCEPTION HANDLING COMPLETED** 🟤

Status : ✓ Completed
Foundation : ✓ Strong
Next Module : 📁 File Handling

"Another Java milestone unlocked! 🏆 Exception Handling — COMPLETED."
